



Complete Concept Map

Interview-Ready Visual Guide

Master All React Concepts with Logical Connections

From Foundation to Production - Complete Learning Path

■ Memory Strategy for Interviews

React Learning Flow:

Foundation → Power Tools → Patterns → Performance → Production

Phase 1 - Fundamentals:

JCPS-ECLFR (JSX, Components, Props, State, Events, Conditional, Lists, Forms, Refs)

Phase 2 - Hooks:

SECRM-LIT-Custom (State, Effect, Context, Ref, Memo, Layout, ImperativeHandle, Transition)

Phase 3 - Patterns:

HRC-PCC (HOC, Render Props, Compound, Provider, Container, Composition)

Phase 4 - Performance:

MLC-VDB (Memo, Lazy/Code-split, Concurrent, Virtualize, Debounce, Bundle)

Phase 5 - State Mgmt:

Local → Context → Global → Server

Interview Mantra:

"Build Foundation → Master Tools → Apply Patterns → Optimize → Test → Deploy"



Master React Journey Map

Foundation

Hooks

Patterns

Performance

State Mgmt

Routing

Data Fetch

Forms & TS

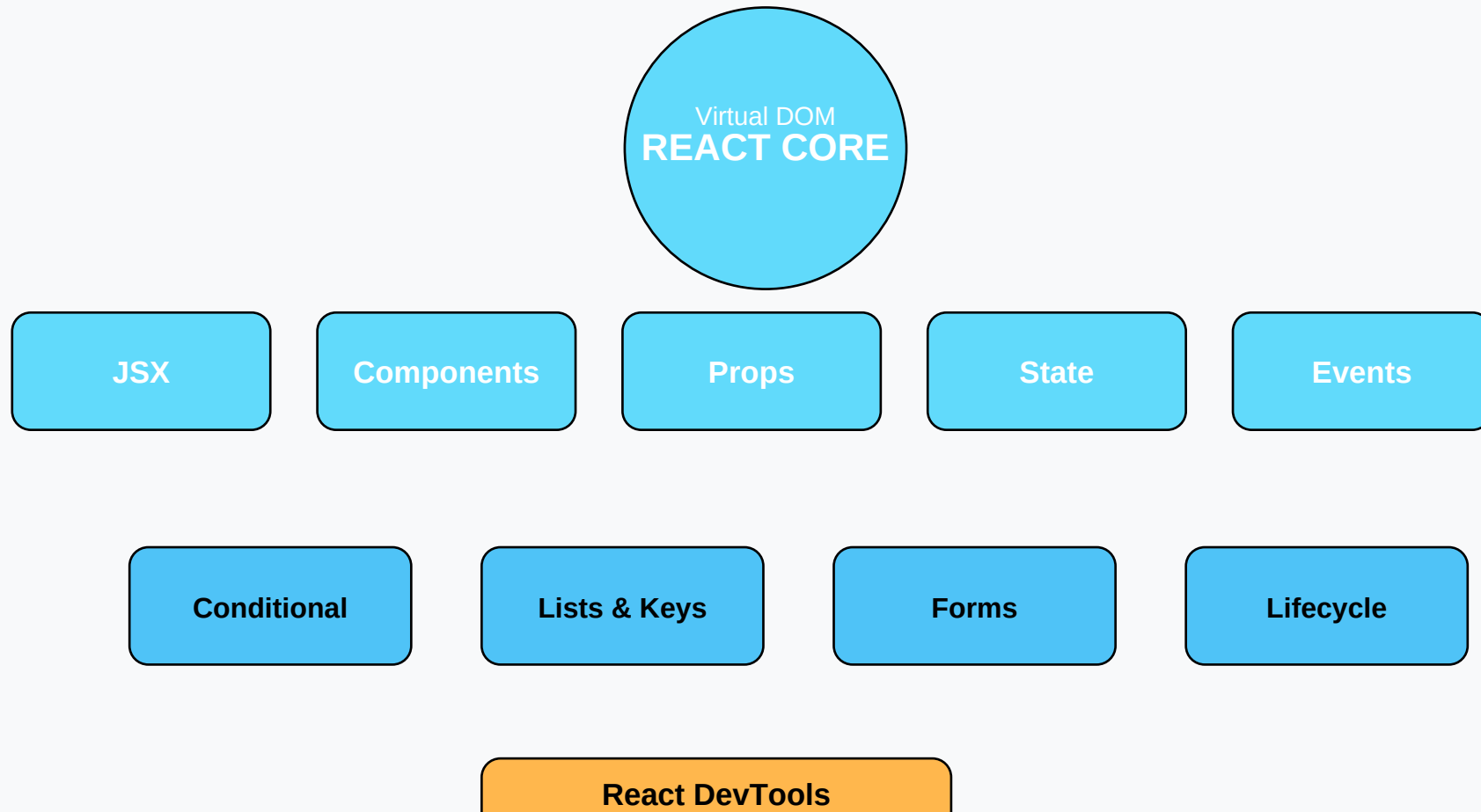
Testing

INTERVIEW MASTERY

System Design • Live Coding • Behavioral • Debugging

Phase 1: Foundation - The React Basics

■ Remember: "JCPS-ECLFR" → JSX, Components, Props, State, Events, Conditional, Lists, Forms, Refs



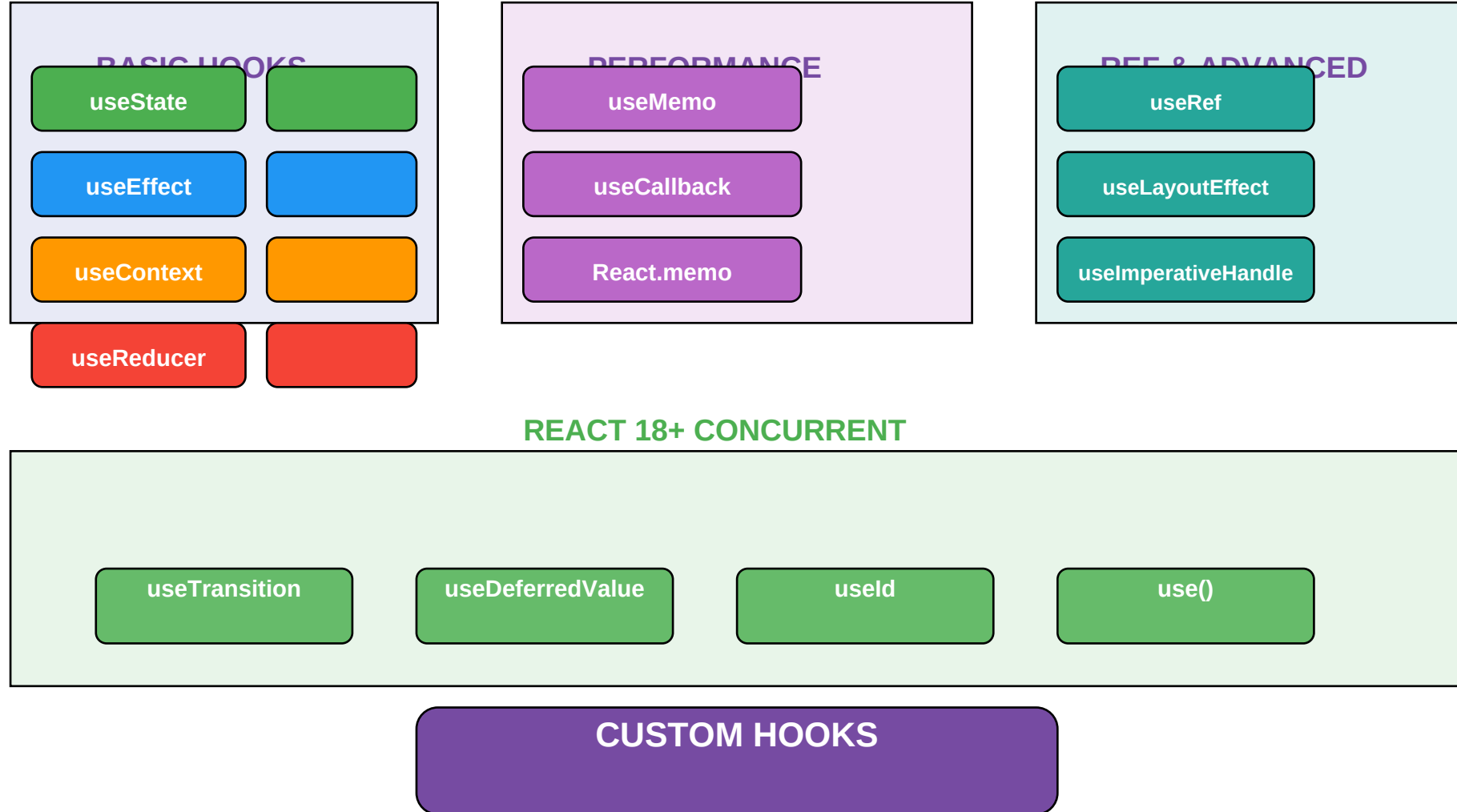
■ Interview Tip: Always connect to Virtual DOM when explaining fundamentals.

Example: "JSX compiles to `React.createElement` which builds the Virtual DOM tree."

Phase 2: Hooks - The Power Tools

■ Remember: "SECRM-LIT-Custom" → State, Effect, Context, Ref, Memo, Layout, ImperativeHandle, Transition

Hook Rules: "Top Level, React Functions Only" (never in loops/conditions)



Phase 3: Design Patterns - Architecture Mastery

■ Remember: "HRC-PCC" → HOC, Render Props, Compound, Provider, Container/Presentational, Composition

PROBLEM: Prop Drilling + Code Reuse

HOC

withAuth()

Render Props

{{(data) => ...}}

Compound

Tab.Panel

Provider

Context API

Container/Presentational

Smart/Dumb

Composition

Children

MODERN: HOOKS + CONTEXT

Most Common Pattern in 2024+

Phase 4: Performance - Speed & Optimization

■ Remember: "MLC-VDB" → Memo, Lazy/Code-split, Concurrent, Virtualize, Debounce, Bundle

RENDER OPTIMIZATION

React.memo

useMemo

useCallback

Concurrent Mode

BUNDLE SIZE

React.lazy

Dynamic Import

Tree Shaking

RUNTIME

Virtualization

Debounce/Throttle

Web Workers

MEASUREMENT TOOLS:

- React DevTools Profiler
- Chrome Performance Tab
- Lighthouse Web Vitals

Phase 5: State Management - Data Flow Mastery

■ Remember: Local → Context → Global (Redux/Zustand) → Server (React Query)

LOCAL

useState, useReducer

Single Component

CONTEXT

useContext, Provider

Shared Nearby

GLOBAL

Redux, Zustand

App-wide

SERVER

React Query, SWR

API Data

■ DECISION FLOWCHART

- Is data from server? → React Query / SWR
- Shared in 2-3 components? → Context API
- Complex app-wide state? → Redux Toolkit
- Simple global state? → Zustand

■ Interview Quick Reference Card

■ Top 5 Must-Know Hooks

1. useState - Component state
2. useEffect - Side effects & lifecycle
3. useContext - Global data access
4. useMemo - Value memoization
5. useCallback - Function memoization

■ Performance Checklist

- React.memo for pure components
- Code splitting with lazy()
- Virtualize long lists
- Debounce search inputs
- Profile before optimizing

■ Common Questions

- Virtual DOM vs Real DOM?
- useEffect cleanup function?
- When to use useCallback?
- Redux vs Context?
- Keys in lists - why?

■ Final Memory Palace:

Ground Floor (Foundation) → 1st Floor (Hooks) → 2nd Floor (Patterns) →
3rd Floor (Performance) → 4th Floor (Production) → Roof (Interview) ■